PROJECT EXPLANATION / 01

Bank marketing

From a business decision to three equivalent models

Explanation and professor-discussion guide

Javier Sánchez Esquivel · Student 19731 · BBADBA 5B
jsanchez.ieu2022@student.ie.edu · 12 October 2026

This learning guide explains the completed project, its measured results, the reasons behind the student-selected decisions, and the concepts needed to discuss it without referring to the code. It is an explicitly AI-generated learning tool, separate from the student-authored interpretation in REPORT.md. Its explanatory prose was revised with Walter Writes and checked for accuracy by Codex.

What was built	Verified outcome
Data and features	41,188 historical bank-marketing records; chronological evaluation; 46 encoded pre-call features.
Three implementations	scikit-learn, manual PyTorch and nn.Module + SGD converge to nearly identical probabilities.
Interactive dashboard	Saved-model inference, model comparison, prediction-versus-actual views, distributions, confusion matrix and threshold costs.
Business result	Test policy cost EUR 17,882; calling everyone costs EUR 16,476. Correct code does not imply a useful policy.
Submission boundary	Fork and upstream PR are prepared separately from Blackboard. See docs/DELIVERY.md for current links and status.

How to study this guide

Begin with the business arithmetic on pages 2 and 8. Next, review the feature definitions on page 4 and the shared model on pages 5-6. Finally, practice the demonstration and questions on pages 10-12. You need to express the tradeoffs in your own words, rather than memorize a script.

PROJECT EXPLANATION / 02

Start with the decision

The student selected a bank calling team that must prioritize customers before a planned call. The target is whether a customer subscribed to a term deposit during the historical campaign. The model estimates a subscription probability, followed by a separate decision about whether to call.

What can be known at that moment?

The implementation includes age; job, marital status, education, default, housing and loan categories; and prior-campaign outcome. It excludes duration, completed-campaign contact count, and final-call channel/month/weekday because the file does not establish their availability before the planned call. I also excluded macroeconomic series because publication-time and vintage information are absent. Although this limits the available predictive signal, it makes the information boundary explicit. Customer profiles and prior-campaign history are assumed to exist in the CRM before a new call.

Economic indicators were excluded conservatively. The dataset provides observed values but no publication-time or revision-vintage information. Excluding them sacrifices potentially useful signal. This is an availability decision, not a claim that macroeconomic conditions are irrelevant.

The student-selected cost assumptions

```
Cost = EUR 2 * (TP + FP) + EUR 50 * FN
```

Term	Business meaning
TP	We call a historical subscriber. This call still costs EUR 2.
FP	We call a historical non-subscriber. The call costs EUR 2.
FN	We skip a historical subscriber. Assign a hypothetical EUR 50 missed-opportunity cost.
TN	We skip a historical non-subscriber. No call cost in this simplified model.

For example, 20 calls and 3 missed subscribers cost 20 × 2 + 3 × 50 = EUR 190. Accuracy is not the primary policy objective. Under these assumptions, a false negative is significantly more costly than an unnecessary call.

A distinction to say clearly

This is response prediction, not causal uplift modeling. Everyone in the historical campaign was exposed to marketing in some form. We cannot observe what would have happened without calling, so we cannot claim the policy creates deposits or measures actual profit.

PROJECT EXPLANATION / 03

Understand the data and split

I chose UCI's Bank Marketing dataset, specifically bank-additional-full.csv. It contains 41,188 client/campaign entries, 20 original inputs and a single binary target, y (term-deposit subscription). According to the supplied data dictionary, the entries are ordered by date from May 2008 through November 2010. Each entry contains customer characteristics and campaign-contact summaries, but exact timestamps and customer identifiers are not provided.

The supplied dictionary indicates that rows are ordered chronologically from May 2008 through November 2010. Exact dates and customer IDs are missing. We maintain the original row order rather than invent dates or sort by month, which would mix different years.

Block	Rows / zero-based IDs	Subscriber rate
Training	24,712 / 0-24,711	4.807%
Validation	8,238 / 24,712-32,949	11.071%
Test	8,238 / 32,950-41,187	30.833%

What each block is allowed to do

During **training**, we learn imputations, numeric means and standard deviations, the one-hot vocabulary and model parameters. During **validation**, we compare a small set of feature and regularization choices and select the threshold. **Testing** evaluates those frozen choices. Adjusting the test threshold to improve the reported result would invalidate that final comparison.

Why chronology matters

The intended use is future campaigns. Randomly mixing early and late records could expose training to future conditions, making chronology more relevant even when results are less flattering. Because exact timestamps are absent, a split boundary may cut through a day or campaign. Separation of repeated customers cannot be guaranteed either.

Data-quality facts

Numeric inputs are median-imputed defensively and standardized using training statistics. One-hot encoding uses the training vocabulary and retains unknown; unseen categories are handled without refitting. There are no blank fields, but some categories are unknown, including 8,597 default and 1,731 education values. The 12 exact duplicate rows are retained because, without identifiers, identical attributes cannot reliably distinguish accidental duplication from distinct customers. No rows were deleted. The 4,110 sentinel/history inconsistencies are documented and preserved. The recency_recorded indicator identifies a recorded day interval; previous and poutcome retain the other history information.

To evaluate later campaigns, I selected the entire ordered version of the file. At 5,834,924 bytes, it is practical to commit and retrain, although its 41,188 rows exceed the assignment's approximate 20k-row guideline. I left the CSV unaltered, with provenance described in data/README.md and its SHA-256 hash in artifacts/metadata.json. There is no runtime data download.

Explain every engineered feature

The original numeric age and categorical customer attributes remain. Categorical values use one-hot encoding. Poutcome retains previous-campaign outcome. The selected feature recipe adds the following:

Feature	Construction	Reason and limitation
recency_recorded	1 if pdays is not 999; otherwise 0	Separates a recorded interval from the 999 sentinel; it does not establish absence of all past contacts.
log_recency_days	log(1 + pdays) when observed; otherwise 0	Compresses elapsed days; the indicator distinguishes the zero placeholder from a real 0-day interval.
log_previous_contacts	log(1 + previous)	Compresses a skewed count. With 0 or 1 previous contact in training, behavior for larger later counts is extrapolation.
loan_combination	housing + / + loan	Represents combinations, including unknown. Allows a joint association beyond separate main effects.
Unknown-field count	Count of unknown among six customer categories	Allows incomplete records to carry a common signal; missingness can change over time.

An example you can explain

If pdays=999 and previous=0, the recency indicator, log recency and log previous contacts are all zero. The sentinel is replaced before the recency log transformation, while log(1 + 0)=0 handles the contact count. If pdays=3 and previous=2, the values are 1, log(4)=1.386 and log(3)=1.099. These values are shown before numeric standardization.

A data-quality caveat: 4,110 records have pdays=999 despite previous>0. The indicator therefore means recency recorded, not never contacted. The count and prior outcome are preserved rather than silently rewriting history.

Preprocessing

Numeric fields are median-imputed defensively and standardized using training statistics: z=(x-training mean)/training standard deviation. No numeric values are missing, so imputation does not alter this dataset. Unknown categories are retained, rather than replaced by a frequent known category. The fitted encoder safely ignores unseen categories, but that does not mean the model understands them.

The selected recipe uses 46 encoded columns. The implementation whitelists allowed inputs, preventing the target and excluded fields from entering the feature matrix. Automated tests perturb forbidden fields and verify that the features remain unchanged.

The model and loss

Each implementation uses **logistic regression**. None includes hidden layers, trees or neural-network feature extractors. PyTorch is a machine-learning framework; using it does not automatically turn a model into deep learning.

```
logit z = x @ w + b
p = sigmoid(z) = 1 / (1 + exp(-z))
call if p >= threshold
```

The weighted sum z can take any real value. The sigmoid maps it to a probability between 0 and 1. The intercept b is the score when numeric inputs are at their training means and the other encoded inputs are zero. With full one-hot encoding, this is not a literal typical customer.

The shared training objective

```
J(w,b) = mean[softplus(z) - y*z] + (lambda/2) * sum(w**2)
softplus(z) = log(1 + exp(z)), computed stably
```

The first term is binary cross-entropy, also called log loss. It penalizes confidently wrong probabilities strongly. The second term, L2 regularization, encourages smaller weights and helps stabilize fitting with redundant or correlated encoded features. The intercept is not regularized. No class weights are assigned; changing them would alter the objective and complicate probability interpretation.

What the gradients mean

```
error = sigmoid(X @ w + b) - y
grad_w = X.T @ error / n + lambda * w
grad_b = mean(error)
```

A gradient tells us how sensitive the objective is to each parameter. Gradient descent subtracts learning_rate × gradient from each parameter. Autograd computes these derivatives from the computation graph. The tests independently compare its gradients with analytical expressions and finite-difference approximations.

Scaling and feature engineering are different

Scaling changes numeric units for optimization and regularization. Feature engineering changes the relationships a model can express. A linear score on transformed inputs can capture log-count effects and category interactions while remaining logistic regression in its parameters.

Three implementations, one objective

Implementation	What happens	What to point to
scikit-learn	LogisticRegression uses L-BFGS to minimize the regularized objective.	train.py: C conversion, fit, predict_proba
Manual PyTorch	Create raw trainable tensors. Compute stable loss, backward, and update tensors inside no_grad. Clear gradients after each step.	train.py: objective and train_torch(standard=False)
Standard PyTorch	nn.Linear produces one logit. BCEWithLogitsLoss and explicit L2 penalty define the loss; optimizer.zero_grad, backward, optimizer.step perform updates.	modeling.py; train_torch(standard=True)

Regularization equivalence is essential

```
PyTorch: mean BCE + lambda/2 * ||w||^2
scikit-learn: mean BCE + 1/(2*C*n) * ||w||^2
Therefore C = 1/(n*lambda) = 0.0040466170
```

Identical numerical values for C and lambda would not produce equivalent objectives. The equivalence test checks that the shared PyTorch objective has a gradient close to zero at scikit-learn's solution.

Learning rate and stopping

Both PyTorch models start with zero weights and zero intercept, use full batches and float64 on CPU, and use the same learning rate 0.790452. It is 1/L, with L computed from a conservative curvature bound on the training matrix. This is a stable training-based step choice, not a search over test scores. Both stop after 6,433 updates when the largest absolute training gradient falls below 1e-7.

Add a column of ones to X for the intercept. For logistic loss, curvature is bounded by 0.25 times the largest eigenvalue of X_aug.T @ X_aug / n. Adding lambda gives a conservative upper bound on curvature for the penalized objective.

Why tiny differences remain

Maximum probability disagreement against scikit-learn is 0.00000561; test call decisions disagree on zero records. Different numerical solvers and finite stopping tolerance account for small differences. Ranking metrics can also shift minutely when nearly tied scores swap order.

Experiments and measured results

This is a small, inspectable comparison of three feature recipes and three L2 strengths (0.001, 0.01 and 0.1), evaluated on the same validation block. The held-out test was not used to make these choices.

Recipe	Best lambda	Validation log loss
raw	0.001	0.369414
history	0.01	0.371278
history_context	0.01	0.370993

I evaluated three feature recipes and three regularization strengths. The raw representation serves as a control that treats 999 as an ordinary number. After reviewing the validation results, I chose the engineered representation with the appropriate sentinel meaning, even though the raw control produced slightly lower validation log loss. The best engineered configuration is selected using validation log loss. I am not claiming that the engineered features improve the measured score over the raw control.

Final test: probability quality

Method	ROC-AUC	Avg precision	Log loss	Brier
sklearn	0.640933	0.485801	0.843136	0.257931
manual	0.640933	0.485804	0.843136	0.257931
standard	0.640933	0.485803	0.843136	0.257931
baseline	0.500000	0.308327	0.969856	0.280993

ROC-AUC measures how often a subscriber ranks above a non-subscriber. **Average precision** summarizes the precision-recall relationship; the constant baseline score here equals the test subscriber rate, 0.308327. **Log loss** measures probability error with strong penalties for confident mistakes. **Brier** measures mean squared probability error.

The naive probability baseline predicts the training prevalence, 0.048074, for each record. At the frozen threshold, it calls everyone. The explicit decision baselines cost EUR 16,476 for call-all and EUR 127,000 for call-none on test. Call-all and call-none are decision policies, not useful probability forecasts. Their full metric rows are included in the export, but their clipped hard-probability log losses should not be used to evaluate probability quality.

Training speed is secondary evidence

This saved run recorded approximately 0.058 seconds for the selected scikit-learn fit, 11.69 seconds for manual PyTorch, and 12.24 seconds for standard PyTorch. These are individual fit timings on this Mac, excluding preprocessing and the small model-selection experiment. They are not a controlled benchmark and will vary.

Thresholds and the business result

The final frozen threshold is **0.03442741**. Validation selects the lowest assumed cost among attainable call policies. Ties favor fewer calls. The implementation groups equal scores and places the boundary between adjacent score levels to avoid floating-point equality problems after saving and loading.

With perfectly calibrated probabilities and correct simplified cost assumptions, calling costs EUR 2 while skipping costs 50 × p in expectation. Calling is therefore preferred when p exceeds 2/50=0.04. The empirical validation threshold differs slightly because validation observations are finite and probability calibration is imperfect.

Policy / test block	Calls	Misses	Assumed cost
Frozen model policy	8,141	32	EUR 17,882
Call everyone	8,238	0	EUR 16,476
Call nobody	0	2,540	EUR 127,000

```
Saved calls = 8,238 - 8,141 = 97
Saved call cost = 97 * EUR 2 = EUR 194
Missed opportunity = 32 * EUR 50 = EUR 1,600
Extra cost vs call-all = EUR 1,406
```

The policy recalls 98.74% of historical subscribers but calls 98.82% of all records. High recall alone does not demonstrate efficient prioritization. Under these costs, missing even a few subscribers can outweigh saving many calls.

Validation versus test

On validation, the selected policy costs EUR 16,444 versus EUR 16,476 for calling everyone: only EUR 32 lower. That small advantage does not generalize to test. The dashboard must expose this outcome, not optimize the test threshold after observing it.

Changing the dashboard costs or threshold is an explanatory what-if exercise. It does not change the saved models or the frozen report results. A new deployed policy would need a separate validation process and later unseen evaluation.

What the evidence cannot establish

Substantial change across chronological blocks

Subscriber prevalence rises from 4.807% to 11.071% to 30.833%. Previous-contact histories also change: training contains only previous=0 or 1, while later blocks contain larger counts. The model therefore extrapolates beyond its training experience. These are observed distribution differences; this project does not establish which changes caused the loss of policy value.

Probability calibration

An AUC above 0.5 shows some ranking signal, but probabilities can still be poorly calibrated. Inspect the predicted-versus-observed plot: points off the diagonal mean predicted probabilities differ from observed subscriber fractions in those bins. Sparse bins have high sampling uncertainty. Do not interpret a bin based on very few examples as a precise estimate.

Scope and missing information

The sample is one bank's historical marketing population from 2008-2010, not current bank customers in general. Exact contact dates, stable customer IDs, cost/revenue measurements and randomized no-call outcomes are absent. A row-order split cannot enforce customer disjointness or a full time gap between campaigns.

Features are hypotheses, not guarantees

The tested engineering improves meaning and explicit handling of missing/sentinel values, but did not beat the raw control on validation log loss. Some training features are rare, including recorded previous-contact intervals. Large later values or different missingness patterns can change model behavior. Coefficients represent conditional associations; correlated inputs and L2 shrinkage prevent causal interpretation.

How a future investigation would remain honest

A follow-up could assess successive later campaign periods, check calibration with a recent validation block, verify the real cost and value assumptions, and obtain customer IDs and exact timestamps. These changes require a later untouched evaluation period. Keep the original test results documented rather than replacing them with tuned results.

Your conclusion belongs to you

The student's supplied conclusion, preserved separately from this guide's teaching explanations:

I would not deploy this policy as is. The most useful next step is a temporal backtest with a calibration check on successive later campaign blocks. Real cost and revenue estimates should replace the hypothetical costs before the policy is reconsidered. The full student reflection and verified arithmetic are preserved in REPORT.md.

Run and demonstrate the project

```
uv sync --frozen
uv run python main.py train
uv run python -m pytest -q
uv run python main.py app
```

Run these from the project folder, then open http://127.0.0.1:7860. The first dependency installation needs Internet access. Training itself reads the included CSV and makes no network requests. Stop the app with Ctrl+C. The required main.py entrypoint remains unchanged.

Demonstration step	What to show and explain
1. Overview	Read the final test result and the cost comparison. Explain why saved call costs do not cover the missed-subscriber penalty.
2. Call policy	Start on validation. Use Call everyone (0%), Call nobody (100%), and Restore saved policy. Switch to test to check later campaigns.
3. Models	Show that metrics agree across three models. Explain calibration and the actual 0/1 outcomes versus predicted probabilities.
4. Data	Show subscriber prevalence by split and a prior-history feature. Explain chronological change and why 999 is a sentinel.
5. Records	Select a record and predict. The app transforms its inputs and invokes loaded model objects; it does not simply fit a new model or train at startup.

Architecture in one sentence

```
local CSV -> train-only preprocessing -> 3 fitted models
-> saved artifacts -> read-only Gradio inference
```

Training stores the preprocessor, scikit-learn model, raw PyTorch tensors, standard PyTorch state_dict and naive baseline. It also exports predictions, metrics, coefficients, split membership, experiment results and training traces. The dashboard reads those files; app.py never calls a fitting routine.

The model pickles are locally created trusted artifacts. Do not replace them with an untrusted downloaded pickle. Rebuilding from the bundled data is the reproducible path.

Verification and grading coverage

Grading component	Evidence to point to
Dataset & business framing: 20%	Pre-call decision, EUR 2/EUR 50 assumptions, chronology, information-availability exclusions; REPORT.md and DECISIONS.md.
Features & preprocessing: 30%	Explicit sentinel semantics, log history, loan interaction, unknown count, training-only transformations; all nine validation configurations retained.
Three-method comparison: 30%	Same objective and data, correct C/lambda mapping, shared PyTorch step/stopping, numerical agreement, naive baseline and candid cost comparison.
Gradio dashboard: 20%	Working interactive views, actual saved models, confusion matrix and business cost respond to threshold changes.
Written comprehension multiplier	Explain both the implementation and its limitations without the code. This guide supports preparation; no grade is guaranteed.

These grading weights are those listed in the repository when inspected; the brief describes them as a proposal subject to confirmation. Both main.py commands must run without manual intervention. REPORT.md supplies evidence for the weighted components rather than receiving a separate grade component.

What the automated checks protect against

The checks cover forbidden-field isolation, sentinel and unknown handling, chronological partitioning and training-only statistics, analytical and finite-difference gradient agreement, objective equivalence, persisted predictions and agreement across methods, threshold endpoints and ties, validation-only threshold selection, inference with fitting disabled, and reproduction of exported dashboard costs.

What local verification does not claim

The project has been trained and tested on this Mac. Browser/API checks verify the local Gradio app. The original GitHub Actions workflow is included. See docs/DELIVERY.md for the latest remote verification status. This is not proof of performance at another bank, causal marketing lift, or a guaranteed grade.

Backup and submission boundary

```
uv run python scripts/package_project.py
```

This creates a local ZIP beside the project folder, excluding the environment, Git internals and caches. It never submits. The authorized GitHub steps are pushing the fork and opening the upstream PR. Providing the PR link and ZIP on Blackboard remains for the student. See docs/DELIVERY.md for links and status.

Practice without your code

Professor question	A complete answer should include
Why logistic regression?	Binary target; a weighted sum mapped through sigmoid; interpretable baseline; assignment asks for the same simple model in three frameworks.
Why is duration leakage?	It is unknown before the call. Good test scores using unavailable inputs would not validate the intended decision.
Why not a random split?	Later-campaign use and changing populations. Preserve source chronology; acknowledge missing exact timestamps and IDs.
What does backward do?	Computes gradients. It does not update weights. Manual no_grad updates or optimizer.step apply the update.
Why zero gradients?	PyTorch accumulates gradients; failing to reset them changes the optimization procedure.
Why BCEWithLogitsLoss?	Stable combination of sigmoid and BCE. Feed logits, not already-sigmoided probabilities.
Why do all models agree?	Same X, y, regularized objective and converged optimization. Their agreement is a correctness check, not a policy-value guarantee.
Why not threshold 0.5?	Decision costs are asymmetric. A subscriber miss costs much more than a call; validation selects the policy threshold.
Why is test accuracy low?	The cost-optimal validation policy calls almost everyone. Accuracy values error types equally; chosen costs do not.
Can you claim feature engineering helped?	It improved explicit semantics; measured validation log loss was slightly worse than the raw control. Preserve that distinction.
Would you deploy?	Give your own recommendation, cite the EUR 1,406 test disadvantage and explain what further evidence you require.

Three short exercises

A. With 100 calls and 4 missed subscribers, what is policy cost? **B.** If lambda doubles at fixed n, what happens to scikit-learn C? **C.** If two frameworks agree exactly, can both be wrong for the business?

Answers: A: EUR 400. B: C halves. C: Yes. Both frameworks may share poor features, a distribution mismatch, invalid business assumptions or an unsuitable decision rule. Agreement alone does not establish business value.

Sources and your final review

Primary sources

1. Assignment brief and rubric: github.com/ami232/daiist-assignment-1. Local copy: docs/ASSIGNMENT.md.

2. UCI Bank Marketing: archive.ics.uci.edu/dataset/222/bank+marketing. Dataset DOI: 10.24432/C5K306. Creators: Moro, Rita and Cortez. License: CC BY 4.0. The local bank-additional-names.txt defines this version's fields.

3. Moro, Cortez and Rita (2014), A Data-Driven Approach to Predict the Success of Bank Telemarketing. Decision Support Systems 62, 22-31. DOI: 10.1016/j.dss.2014.03.001.

4. scikit-learn logistic-regression objective: scikit-learn.org/stable/modules/linear_model.html#logistic-regression. PyTorch loss implementation: BCEWithLogitsLoss and functional.softplus in the installed torch package.

Evidence generated by this project

All numerical results in this guide come from artifacts/metadata.json, metrics.csv, feature_experiments.csv and the committed source data. They are not borrowed benchmark results. The exact source hash is:

```
74adfc578bf77a7ff4bb1ba4a9f8709d
9e3c6907342959c2c8416847e0afb4d8
```

Package	Saved run version
numpy	2.5.3
pandas	3.0.6
scikit-learn	1.9.1
torch	2.11.0
gradio	6.28.0

Before you explain the project

Reconstruct the EUR 1,406 comparison, explain why the cost-based threshold is low, derive C=1/(n*lambda), and identify which information was excluded from the pre-call decision. Practice the five-step dashboard demonstration. Check that the report conclusion reflects what you personally think.

AI disclosure and ownership

Codex implemented and debugged the code, tests and dashboard, formatted evidence, and authored this AI-generated guide. The student selected the topic, decision point, cost assumptions, split strategy, feature ideas and semantic tradeoff, and supplied the recommendation. Walter Writes revised prose, with factual corrections by Codex. The student remains responsible for understanding the work.

Blackboard submission has not been performed. The GitHub fork and PR are authorized preparation steps; the current links and status are recorded in docs/DELIVERY.md. The ZIP is a local copy for the student to upload.